

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 3 Practical No: 7

Roll no: 35 PRN No: 12414739 Name of Student: KHYATI KENDHE

Subject Name : Deep Learning

Problem Statement : Implement transfer learning using pre-trained AlexNet, VGG16, ResNet50, and EfficientNetB0 models for image classification, and compare their performance.

Algorithm :

- **Load & reduce CIFAR-10**
Load the CIFAR-10 dataset (50k train / 10k test, 32×32 RGB images, 10 classes). Reduce to 20,000 training and 5,000 test images to keep runtime manageable. Flatten label arrays from (n,1) to (n,).
- **Preprocess images**
Resize every image from 32×32 to 96×96 using `tf.image.resize`, since the pretrained CNNs expect larger inputs. Cast the resized arrays to float32.
- **Build AlexNet from scratch**
Define a custom AlexNet-style CNN: 5 convolutional blocks (with batch normalization and max-pooling) followed by two dense layers with dropout, ending in a 10-way softmax output. Compile with the Adam optimizer and sparse categorical crossentropy loss.
- **Train & evaluate AlexNet**
Train AlexNet end-to-end on the resized, normalized ($\div 255$) images for 3 epochs, recording training time. Plot accuracy/loss curves and evaluate final test accuracy and loss.
- **Extract features from pretrained CNNs**
For VGG16, ResNet50, and EfficientNetB0: freeze the pretrained ImageNet weights (`trainable = False`), apply each model's own `preprocess_input` function, and pass the images through to obtain fixed feature maps — no backbone training involved.

- **Train a classifier head per model**
Attach a small trainable head to each set of extracted features: GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.5) → Dense(10, softmax). Train each head for 3 epochs on its corresponding features. This is transfer learning — far cheaper than fine-tuning the whole backbone.
- **Compare all four models**
Collect test accuracy and training time for AlexNet, VGG16, ResNet50, and EfficientNetB0 into a single results table. Visualize with bar charts (accuracy, training time) and overlaid training/validation accuracy curves.
- **Diagnose errors per model**
Generate predictions for each model. Produce a confusion matrix and classification report (precision, recall, F1 per class) for each. Finish with a grid of sample test images showing true vs. predicted labels (using EfficientNetB0, the best performer).
-

Code :

```
import subprocess
import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.applications import (
    VGG16,
    ResNet50,
    EfficientNetB0
)
from tensorflow.keras.applications.vgg16 import preprocess_input as vgg_preprocess
from tensorflow.keras.applications.resnet50 import preprocess_input as resnet_preprocess
from tensorflow.keras.applications.efficientnet import preprocess_input as efficientnet_preprocess

from sklearn.metrics import (
    confusion_matrix,
    classification_report
)

print("TensorFlow version:", tf.__version__)
print("GPU Available:", tf.config.list_physical_devices('GPU'))

# Optional: show GPU details if nvidia-smi is available (Colab-only
# command replaced with a script-safe equivalent).
try:
    print(subprocess.run(["nvidia-smi"], capture_output=True, text=True).stdout)
except FileNotFoundError:
    print("nvidia-smi not available on this machine — skipping.")
```

```

# =====
# LOAD CIFAR-10
# =====

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

print("\nOriginal Dataset")
print("Training:", x_train.shape)
print("Testing :", x_test.shape)

class_names = [
    "airplane",
    "automobile",
    "bird",
    "cat",
    "deer",
    "dog",
    "frog",
    "horse",
    "ship",
    "truck"
]

# =====
# REDUCE CIFAR-10 DATASET
# =====

TRAIN_SAMPLES = 20000
TEST_SAMPLES = 5000

x_train = x_train[:TRAIN_SAMPLES]
y_train = y_train[:TRAIN_SAMPLES]

x_test = x_test[:TEST_SAMPLES]
y_test = y_test[:TEST_SAMPLES]

# Convert labels from (n,1) to (n,)
y_train = y_train.flatten()
y_test = y_test.flatten()

print("\n")
print("=" * 60)
print("REDUCED DATASET")
print("=" * 60)

print("Training images:", x_train.shape)
print("Testing images :", x_test.shape)

plt.figure(figsize=(12, 8))

for i in range(12):
    plt.subplot(3, 4, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i]])
    plt.axis("off")

```

```
plt.suptitle("Sample CIFAR-10 Images", fontsize=16)
plt.tight_layout()
plt.show()
```

```
IMG_SIZE = (96, 96)
```

```
x_train_resized = tf.image.resize(x_train, IMG_SIZE)
x_test_resized = tf.image.resize(x_test, IMG_SIZE)
```

```
x_train_resized = tf.cast(x_train_resized, tf.float32)
x_test_resized = tf.cast(x_test_resized, tf.float32)
```

```
print("Resized training data:", x_train_resized.shape)
print("Resized testing data :", x_test_resized.shape)
```

```
def create_alexnet():
```

```
    model = models.Sequential([

        layers.Conv2D(96, (11, 11), strides=4,
                      activation='relu',
                      input_shape=(96, 96, 3)),
        layers.BatchNormalization(),
        layers.MaxPooling2D((3, 3), strides=2),

        layers.Conv2D(256, (5, 5), padding='same',
                      activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((3, 3), strides=2),

        layers.Conv2D(384, (3, 3), padding='same',
                      activation='relu'),

        layers.Conv2D(384, (3, 3), padding='same',
                      activation='relu'),

        layers.Conv2D(256, (3, 3), padding='same',
                      activation='relu'),
        layers.MaxPooling2D((3, 3), strides=2),

        layers.Flatten(),

        layers.Dense(512, activation='relu'),
        layers.Dropout(0.5),

        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5),

        layers.Dense(10, activation='softmax')
    ])
```

```
    return model
```

```
alexnet = create_alexnet()
```

```
alexnet.summary()
```

```
alexnet.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
EPOCHS = 3  
BATCH_SIZE = 64
```

```
start_time = time.time()
```

```
alex_history = alexnet.fit(  
    x_train_resized / 255.0,  
    y_train,  
    validation_data=(x_test_resized / 255.0, y_test),  
    epochs=EPOCHS,  
    batch_size=BATCH_SIZE,  
    verbose=1  
)
```

```
alex_time = time.time() - start_time
```

```
print(f"\nAlexNet Training Time: {alex_time:.2f} seconds")
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)  
plt.plot(alex_history.history['accuracy'], marker='o', label='Training Accuracy')  
plt.plot(alex_history.history['val_accuracy'], marker='o', label='Validation Accuracy')  
plt.title("AlexNet Accuracy")  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.legend()  
plt.grid(True)
```

```
plt.subplot(1, 2, 2)  
plt.plot(alex_history.history['loss'], marker='o', label='Training Loss')  
plt.plot(alex_history.history['val_loss'], marker='o', label='Validation Loss')  
plt.title("AlexNet Loss")  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.legend()  
plt.grid(True)
```

```
plt.tight_layout()  
plt.show()
```

```
alex_loss, alex_accuracy = alexnet.evaluate(  
    x_test_resized / 255.0,  
    y_test,  
    verbose=1
```

```

)

print("\nAlexNet Test Accuracy:", round(alex_accuracy * 100, 2), "%")
print("AlexNet Test Loss:", round(alex_loss, 4))

def extract_features(base_model, images, preprocess_function):

    processed_images = preprocess_function(images)

    features = base_model.predict(
        processed_images,
        batch_size=64,
        verbose=1
    )

    return features

vgg_base = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(96, 96, 3)
)

vgg_base.trainable = False
vgg_base.summary()

start_time = time.time()

vgg_train_features = extract_features(vgg_base, x_train_resized, vgg_preprocess)
vgg_test_features = extract_features(vgg_base, x_test_resized, vgg_preprocess)

vgg_feature_time = time.time() - start_time

print("VGG16 training features:", vgg_train_features.shape)
print("VGG16 testing features :", vgg_test_features.shape)
print(f"Feature extraction time: {vgg_feature_time:.2f} seconds")

vgg_classifier = models.Sequential([
    layers.Input(shape=vgg_train_features.shape[1:]),
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

vgg_classifier.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

vgg_classifier.summary()

```

```

start_time = time.time()

vgg_history = vgg_classifier.fit(
    vgg_train_features,
    y_train,
    validation_data=(vgg_test_features, y_test),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)

vgg_time = time.time() - start_time

print(f"\nVGG16 Training Time: {vgg_time:.2f} seconds")

resnet_base = ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(96, 96, 3)
)

resnet_base.trainable = False
resnet_base.summary()

start_time = time.time()

resnet_train_features = extract_features(resnet_base, x_train_resized, resnet_preprocess)
resnet_test_features = extract_features(resnet_base, x_test_resized, resnet_preprocess)

resnet_feature_time = time.time() - start_time

print("ResNet50 training features:", resnet_train_features.shape)
print("ResNet50 testing features:", resnet_test_features.shape)
print(f"Feature extraction time: {resnet_feature_time:.2f} seconds")

resnet_classifier = models.Sequential([
    layers.Input(shape=resnet_train_features.shape[1:]),
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

resnet_classifier.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

resnet_classifier.summary()

start_time = time.time()

```

```

resnet_history = resnet_classifier.fit(
    resnet_train_features,
    y_train,
    validation_data=(resnet_test_features, y_test),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)

```

```

resnet_time = time.time() - start_time

```

```

print(f"\nResNet50 Training Time: {resnet_time:.2f} seconds")

```

```

efficient_base = EfficientNetB0(
    weights='imagenet',
    include_top=False,
    input_shape=(96, 96, 3)
)

```

```

efficient_base.trainable = False
efficient_base.summary()

```

```

start_time = time.time()

```

```

efficient_train_features = extract_features(efficient_base, x_train_resized, efficientnet_preprocess)
efficient_test_features = extract_features(efficient_base, x_test_resized, efficientnet_preprocess)

```

```

efficient_feature_time = time.time() - start_time

```

```

print("EfficientNetB0 training features:", efficient_train_features.shape)
print("EfficientNetB0 testing features:", efficient_test_features.shape)
print(f"Feature extraction time: {efficient_feature_time:.2f} seconds")

```

```

efficient_classifier = models.Sequential([
    layers.Input(shape=efficient_train_features.shape[1:]),
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

```

```

efficient_classifier.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

```

efficient_classifier.summary()

```

```

start_time = time.time()

```

```

efficient_history = efficient_classifier.fit(
    efficient_train_features,
    y_train,
    validation_data=(efficient_test_features, y_test),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)

efficient_time = time.time() - start_time

print(f"\nEfficientNetB0 Training Time: {efficient_time:.2f} seconds")

results = pd.DataFrame({
    'Model': [
        'AlexNet',
        'VGG16',
        'ResNet50',
        'EfficientNetB0'
    ],
    'Test Accuracy': [
        alex_accuracy,
        vgg_classifier.evaluate(vgg_test_features, y_test, verbose=0)[1],
        resnet_classifier.evaluate(resnet_test_features, y_test, verbose=0)[1],
        efficient_classifier.evaluate(efficient_test_features, y_test, verbose=0)[1]
    ],
    'Training Time (sec)': [
        alex_time,
        vgg_time,
        resnet_time,
        efficient_time
    ]
})

results['Test Accuracy (%)'] = results['Test Accuracy'] * 100

print(results)

plt.figure(figsize=(10, 6))

bars = plt.bar(results['Model'], results['Test Accuracy (%)'])

plt.title("Model Accuracy Comparison", fontsize=16)
plt.xlabel("Model")
plt.ylabel("Test Accuracy (%)")
plt.ylim(0, 100)

for bar, value in zip(bars, results['Test Accuracy (%)']):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 1,
        f"{value:.2f}%",
    )

```

```

        ha='center',
        fontsize=11
    )

plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))

bars = plt.bar(results['Model'], results['Training Time (sec)'])

plt.title("Training Time Comparison", fontsize=16)
plt.xlabel("Model")
plt.ylabel("Training Time (seconds)")

for bar, value in zip(bars, results['Training Time (sec)']):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height(),
        f'{value:.1f}s',
        ha='center',
        va='bottom'
    )

plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))

plt.plot(alex_history.history['accuracy'], marker='o', label='AlexNet')
plt.plot(vgg_history.history['accuracy'], marker='o', label='VGG16')
plt.plot(resnet_history.history['accuracy'], marker='o', label='ResNet50')
plt.plot(efficient_history.history['accuracy'], marker='o', label='EfficientNetB0')

plt.title("Training Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))

plt.plot(alex_history.history['val_accuracy'], marker='o', label='AlexNet')
plt.plot(vgg_history.history['val_accuracy'], marker='o', label='VGG16')
plt.plot(resnet_history.history['val_accuracy'], marker='o', label='ResNet50')
plt.plot(efficient_history.history['val_accuracy'], marker='o', label='EfficientNetB0')

plt.title("Validation Accuracy Comparison")

```

```
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```

```
alex_pred = np.argmax(alexnet.predict(x_test_resized / 255.0, verbose=0), axis=1)
vgg_pred = np.argmax(vgg_classifier.predict(vgg_test_features, verbose=0), axis=1)
resnet_pred = np.argmax(resnet_classifier.predict(resnet_test_features, verbose=0), axis=1)
efficient_pred = np.argmax(efficient_classifier.predict(efficient_test_features, verbose=0), axis=1)
```

```
predictions = {
    'AlexNet': alex_pred,
    'VGG16': vgg_pred,
    'ResNet50': resnet_pred,
    'EfficientNetB0': efficient_pred
}
```

```
for model_name, pred in predictions.items():
```

```
    cm = confusion_matrix(y_test, pred)
```

```
    plt.figure(figsize=(8, 6))
```

```
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        xticklabels=class_names,
        yticklabels=class_names
    )
```

```
    plt.title(f'{model_name} - Confusion Matrix')
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
```

```
    plt.tight_layout()
    plt.show()
```

```
for model_name, pred in predictions.items():
```

```
    print("\n")
    print("=" * 70)
    print(model_name)
    print("=" * 70)
```

```
    print(classification_report(y_test, pred, target_names=class_names))
```

```
plt.figure(figsize=(15, 10))
```

```
for i in range(12):
```

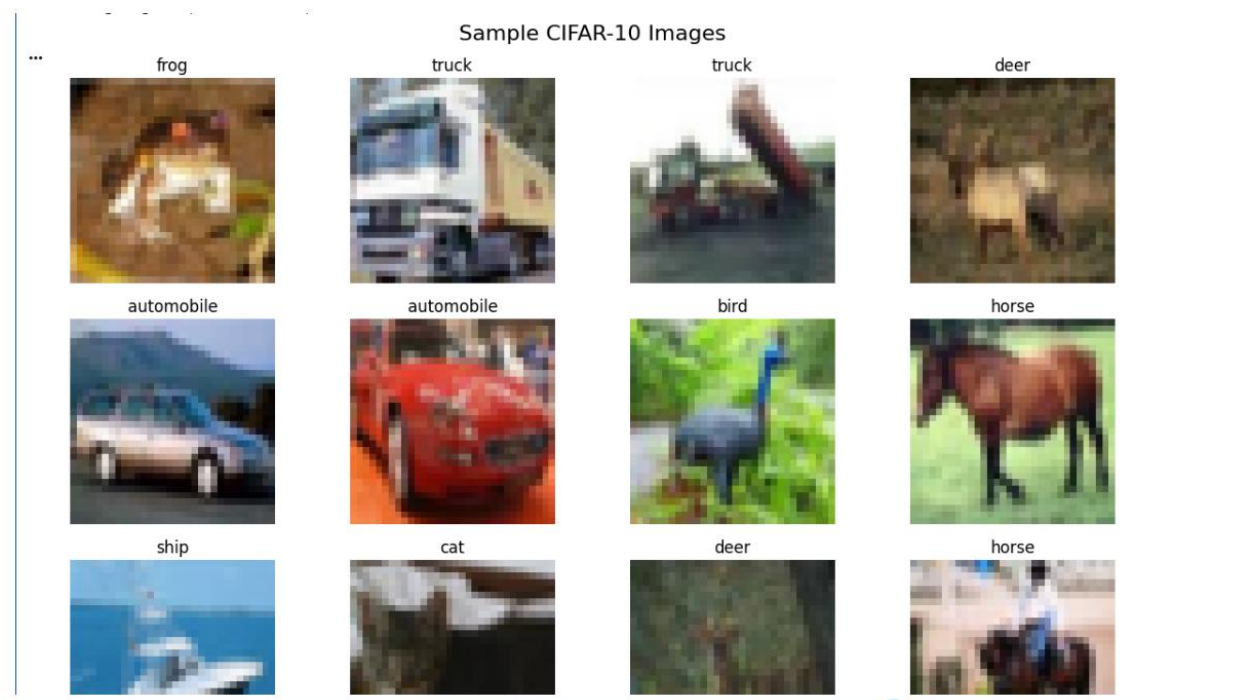
```
plt.subplot(3, 4, i + 1)
plt.imshow(x_test[i])

true_label = class_names[y_test[i]]
predicted_label = class_names[efficient_pred[i]]

plt.title(f"True: {true_label}\nPred: {predicted_label}")
plt.axis("off")

plt.suptitle("EfficientNetB0 Sample Predictions", fontsize=16)
plt.tight_layout()
plt.show()
```

Output :



```

... Resized training data: (20000, 96, 96, 3)
Resized testing data : (5000, 96, 96, 3)
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape` argument to `Conv2D` layers.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

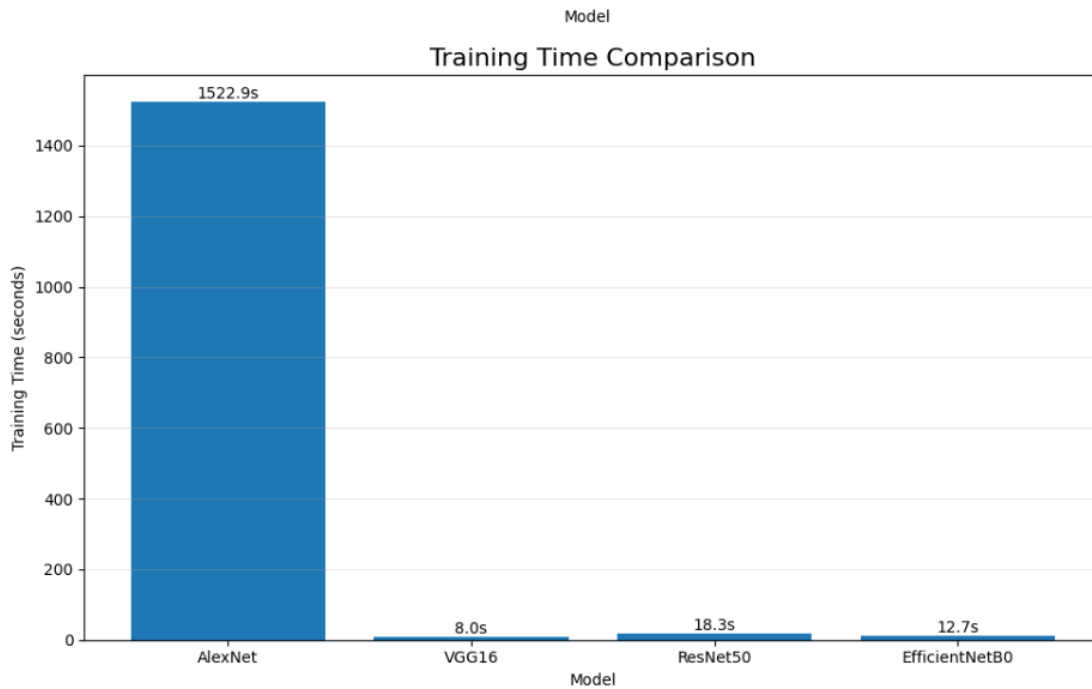
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 22, 22, 96)	34,944
batch_normalization (BatchNormalization)	(None, 22, 22, 96)	384
max_pooling2d (MaxPooling2D)	(None, 10, 10, 96)	0
conv2d_1 (Conv2D)	(None, 10, 10, 256)	614,656
batch_normalization_1 (BatchNormalization)	(None, 10, 10, 256)	1,024
max_pooling2d_1 (MaxPooling2D)	(None, 4, 4, 256)	0
conv2d_2 (Conv2D)	(None, 4, 4, 384)	885,120
conv2d_3 (Conv2D)	(None, 4, 4, 384)	1,327,488
conv2d_4 (Conv2D)	(None, 4, 4, 256)	884,992
max_pooling2d_2 (MaxPooling2D)	(None, 1, 1, 256)	0
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 512)	131,584
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 10)	2,570

```

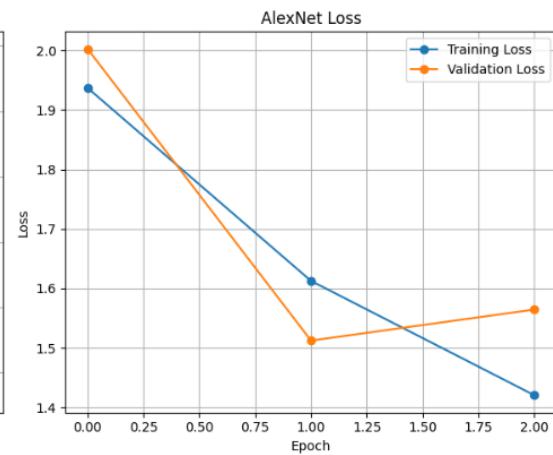
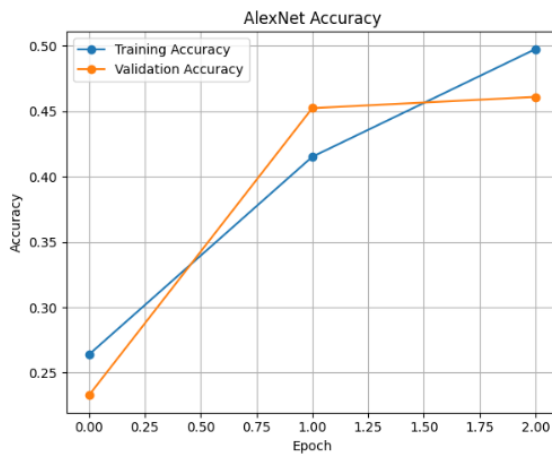
Total params: 4,014,090 (15.31 MB)
Trainable params: 4,013,386 (15.31 MB)
Non-trainable params: 704 (2.75 KB)
Epoch 1/3
313/313 ————— 498s 2s/step - accuracy: 0.2639 - loss: 1.9368 - val_accuracy: 0.2328 - val_loss: 2.0022
Epoch 2/3

```



Epoch 1/3
 313/313 — 498s 2s/step - accuracy: 0.2639 - loss: 1.9368 - val_accuracy: 0.2328 - val_loss: 2.0022
 Epoch 2/3
 313/313 — 517s 2s/step - accuracy: 0.4152 - loss: 1.6126 - val_accuracy: 0.4524 - val_loss: 1.5123
 Epoch 3/3
 313/313 — 506s 2s/step - accuracy: 0.4976 - loss: 1.4205 - val_accuracy: 0.4610 - val_loss: 1.5645

AlexNet Training Time: 1522.86 seconds



157/157 — 29s 181ms/step - accuracy: 0.4610 - loss: 1.5645

AlexNet Test Accuracy: 46.1 %

AlexNet Test Loss: 1.5645

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5

58889256/58889256 — 0s 0us/step

Model: "vgg16"

AlexNet Test Loss: 1.5645
 Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
 58889256/58889256 ————— 0s 0us/step
 ...
 Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 96, 96, 3)	0
block1_conv1 (Conv2D)	(None, 96, 96, 64)	1,792
block1_conv2 (Conv2D)	(None, 96, 96, 64)	36,928
block1_pool (MaxPooling2D)	(None, 48, 48, 64)	0
block2_conv1 (Conv2D)	(None, 48, 48, 128)	73,856
block2_conv2 (Conv2D)	(None, 48, 48, 128)	147,584
block2_pool (MaxPooling2D)	(None, 24, 24, 128)	0
block3_conv1 (Conv2D)	(None, 24, 24, 256)	295,168
block3_conv2 (Conv2D)	(None, 24, 24, 256)	590,080
block3_conv3 (Conv2D)	(None, 24, 24, 256)	590,080
block3_pool (MaxPooling2D)	(None, 12, 12, 256)	0
block4_conv1 (Conv2D)	(None, 12, 12, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 12, 12, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 12, 12, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 6, 6, 512)	0
block5_conv1 (Conv2D)	(None, 6, 6, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 6, 6, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 6, 6, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 3, 3, 512)	0

...

block5_conv2 (Conv2D)	(None, 6, 6, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 6, 6, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 3, 3, 512)	0

Total params: 14,714,688 (56.13 MB)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 14,714,688 (56.13 MB)
 313/313 ————— 2186s 7s/step
 79/79 ————— 547s 7s/step
 VGG16 training features: (20000, 3, 3, 512)
 VGG16 testing features : (5000, 3, 3, 512)
 Feature extraction time: 2802.11 seconds
 Model: "sequential_1"

Layer (type)	Output Shape	Param #
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense_3 (Dense)	(None, 128)	65,664
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 10)	1,290

Total params: 66,954 (261.54 KB)
 Trainable params: 66,954 (261.54 KB)
 Non-trainable params: 0 (0.00 B)
 Epoch 1/3
 313/313 ————— 4s 9ms/step - accuracy: 0.5479 - loss: 1.8086 - val_accuracy: 0.7382 - val_loss: 0.7832
 Epoch 2/3
 313/313 ————— 2s 6ms/step - accuracy: 0.6989 - loss: 0.9011 - val_accuracy: 0.7776 - val_loss: 0.6503
 Epoch 3/3
 313/313 ————— 2s 5ms/step - accuracy: 0.7483 - loss: 0.7456 - val_accuracy: 0.7872 - val_loss: 0.6173

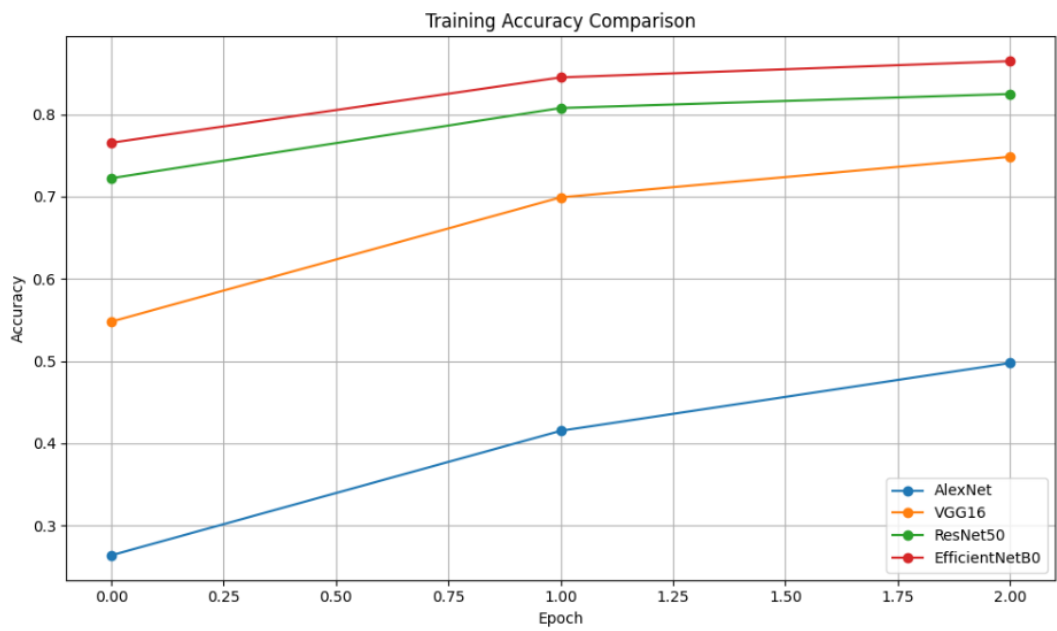
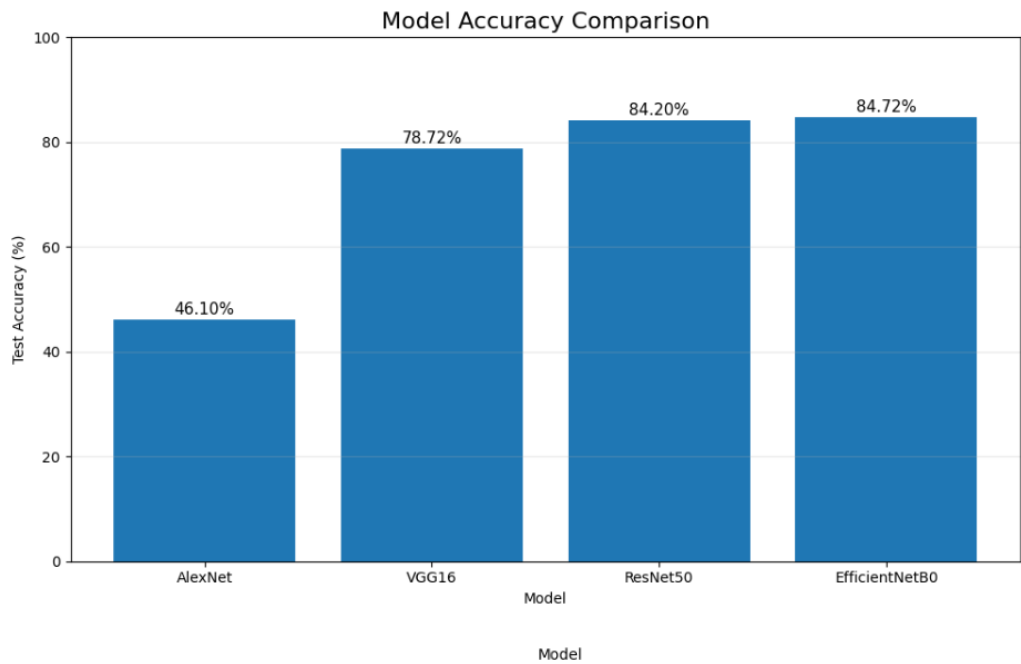
VGG16 Training Time: 8.00 seconds

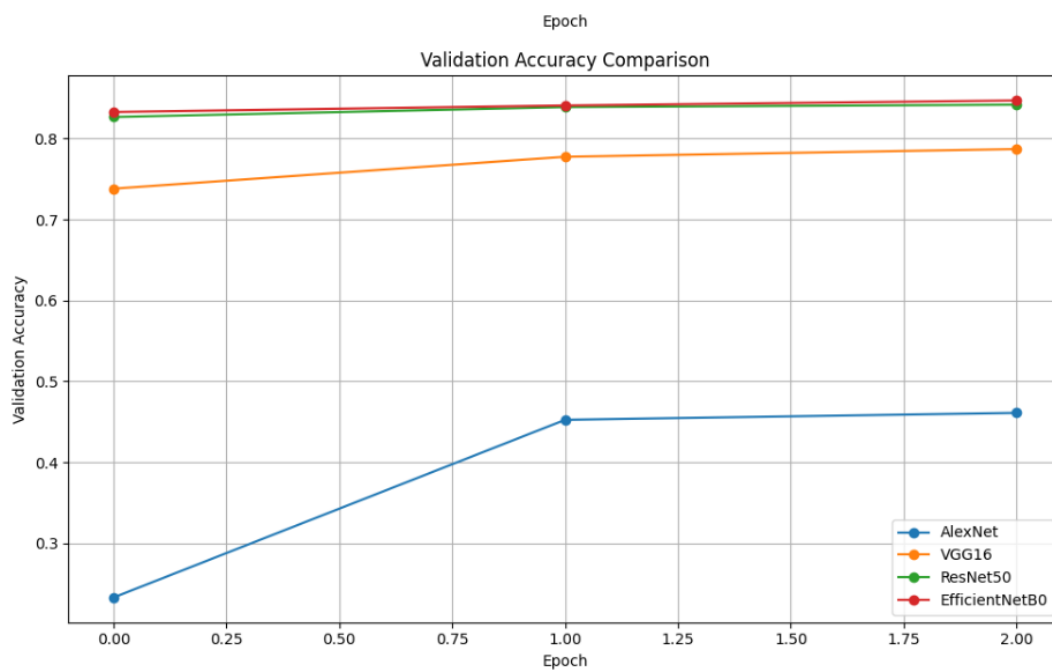
```

12.7322 0.8472 12.7322 accuracy: 0.8472 loss: 0.4700 val_accuracy: 0.8472 val_loss: 0.4700
... EfficientNetB0 Training Time: 12.73 seconds

```

Model	Test Accuracy	Training Time (sec)	Test Accuracy (%)
0 AlexNet	0.4610	1522.860675	46.100000
1 VGG16	0.7872	7.998814	78.719997
2 ResNet50	0.8420	18.268876	84.200001
3 EfficientNetB0	0.8472	12.732499	84.719998





```

=====
AlexNet
...
=====
precision    recall  f1-score   support

  airplane    0.42    0.57    0.49     488
  automobile    0.67    0.58    0.62     505
    bird    0.31    0.56    0.40     512
    cat    0.32    0.38    0.35     497
    deer    0.54    0.19    0.28     507
    dog    0.40    0.35    0.38     488
    frog    0.75    0.37    0.50     491
    horse    0.74    0.47    0.58     495
    ship    0.40    0.80    0.53     504
    truck    0.77    0.34    0.47     513

 accuracy          0.46    5000
  macro avg    0.53    0.46    0.46    5000
 weighted avg    0.53    0.46    0.46    5000

```

```

=====
VGG16
=====
precision    recall  f1-score   support

  airplane    0.75    0.86    0.80     488
  automobile    0.87    0.90    0.88     505
    bird    0.76    0.71    0.74     512
    cat    0.70    0.54    0.61     497
    deer    0.74    0.73    0.73     507
    dog    0.71    0.74    0.72     488
    frog    0.77    0.85    0.81     491
    horse    0.81    0.85    0.83     495
    ship    0.86    0.87    0.86     504
    truck    0.89    0.83    0.86     513

 accuracy          0.79    5000
  macro avg    0.79    0.79    0.78    5000
 weighted avg    0.79    0.79    0.78    5000

```

```

=====
... ResNet50
=====
      precision    recall  f1-score   support

   airplane      0.89      0.81      0.85      488
  automobile      0.91      0.90      0.90      505
         bird      0.83      0.82      0.82      512
          cat      0.69      0.72      0.70      497
         deer      0.79      0.79      0.79      507
          dog      0.84      0.75      0.79      488
         frog      0.86      0.90      0.88      491
         horse      0.86      0.89      0.88      495
          ship      0.86      0.94      0.90      504
         truck      0.90      0.89      0.89      513

 accuracy              0.84      5000
  macro avg           0.84      0.84      0.84      5000
 weighted avg           0.84      0.84      0.84      5000

```

```

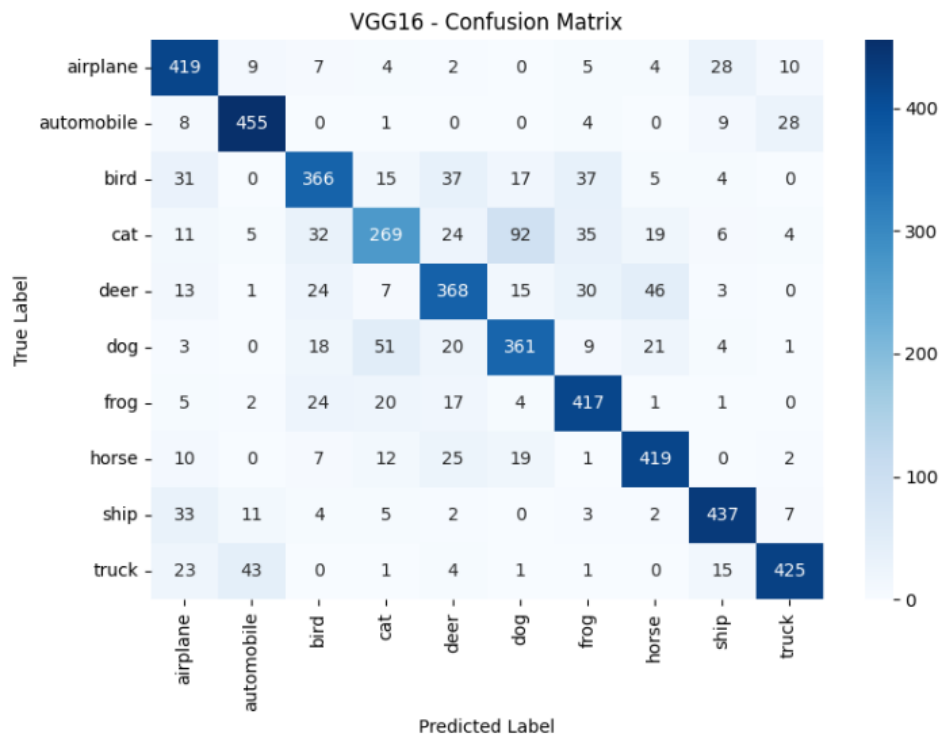
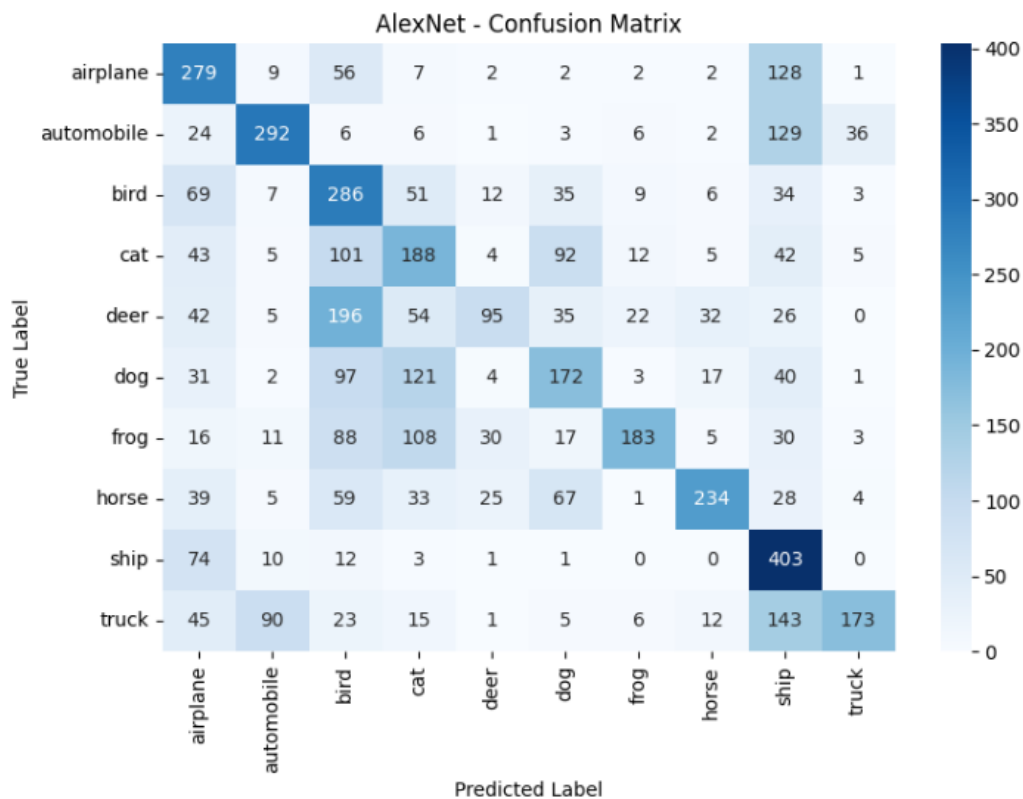
=====
EfficientNetB0
=====
      precision    recall  f1-score   support

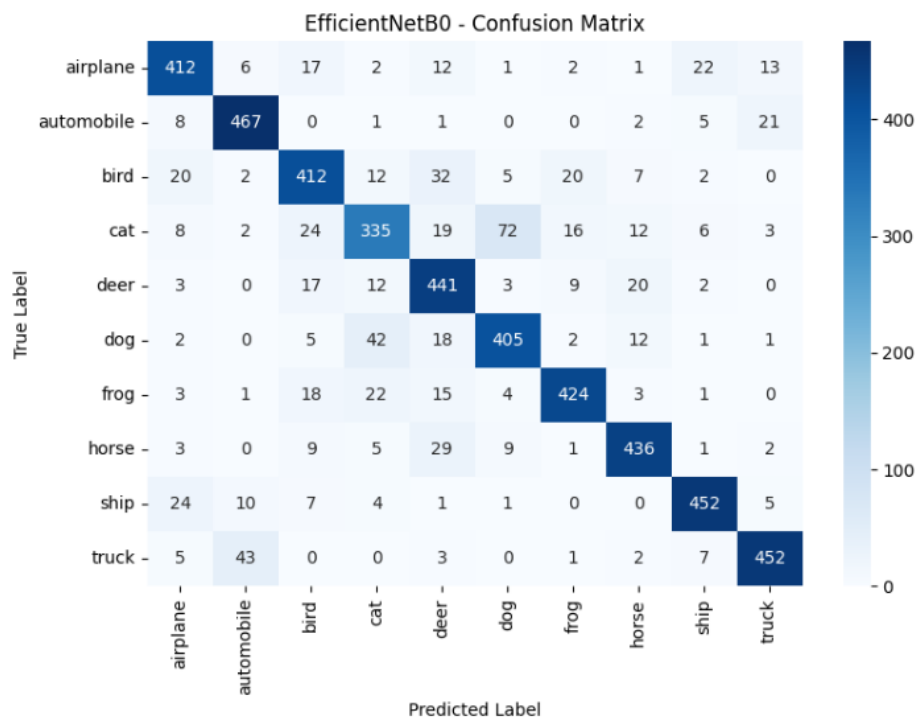
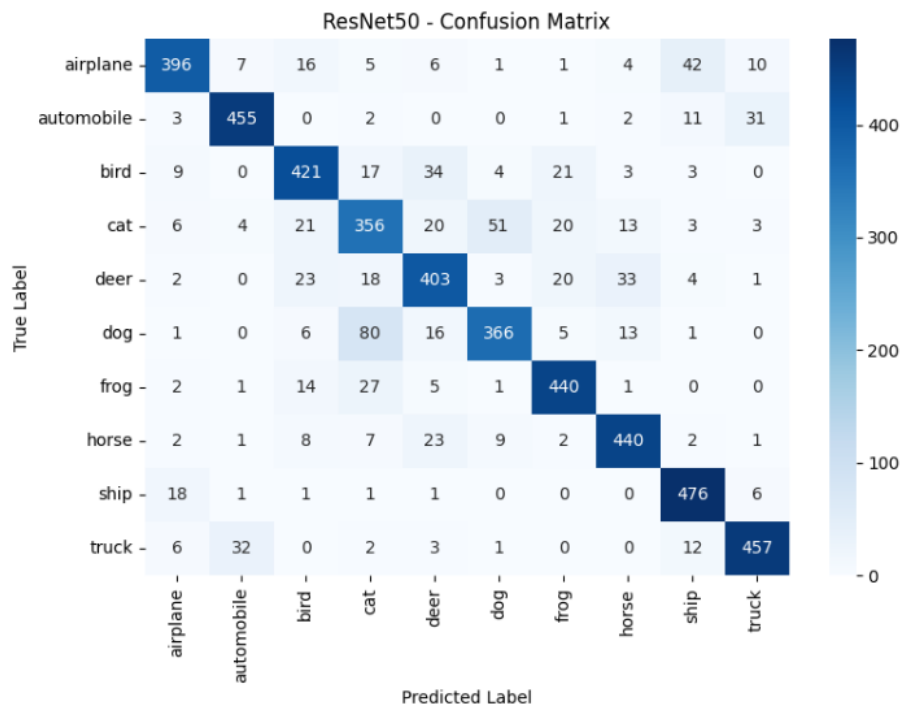
   airplane      0.84      0.84      0.84      488
  automobile      0.88      0.92      0.90      505
         bird      0.81      0.80      0.81      512
          cat      0.77      0.67      0.72      497
         deer      0.77      0.87      0.82      507
          dog      0.81      0.83      0.82      488
         frog      0.89      0.86      0.88      491
         horse      0.88      0.88      0.88      495
          ship      0.91      0.90      0.90      504
         truck      0.91      0.88      0.90      513

 accuracy              0.85      5000
  macro avg           0.85      0.85      0.85      5000
 weighted avg           0.85      0.85      0.85      5000

```

...

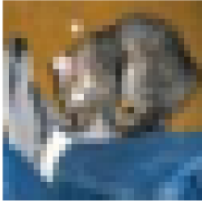




...

EfficientNetB0 Sample Predictions

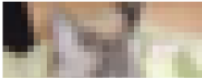
True: cat
Pred: cat



True: frog
Pred: frog



True: cat
Pred: cat



True: ship
Pred: ship



True: frog
Pred: frog



True: automobile
Pred: automobile



True: ship
Pred: automobile



True: automobile
Pred: automobile



True: airplane
Pred: airplane



True: airplane
Pred: truck



True: frog
Pred: frog



True: truck
Pred: truck



